



GUIA PRÁTICO PASSO A PASSO

7 Projetos com Apache Kafka e Java

Do produtor/consumidor básico às transações:
estrutura de pastas, dependências, código
esqueleto e checklist de teste para cada tópico

Java 8 • 11 • 17 • 21 • 25

Docker Compose • Maven • kafka-clients

Partições • Consumer Groups • Chaves • Retry • Transações • Graceful Shutdown

Sumário

- 0. Como usar este guia
- 1. Projeto 1 — Hello World (Java 8)
- 2. Projeto 2 — Partições e Consumer Group (Java 11)
- 3. Projeto 3 — Múltiplos Consumer Groups Independentes (Java 17)
- 4. Projeto 4 — Chaves e Ordenação (Java 21)
- 5. Projeto 5 — Falhas, Retry e Reprocessamento (Java 25)
- 6. Projeto 6 — Exactly-Once com Transações (Java 21)
- 7. Projeto 7 — Consumer Produtivo (Java 17)
- 8. Infraestrutura comum (Docker Compose único)

0. Como usar este guia

Assim como no guia de RabbitMQ, cada projeto abaixo é **independente** — pode ter sua própria pasta, seu próprio `pom.xml`, e rodar sozinho. A sugestão é seguir a ordem, porque cada um introduz um conceito novo em cima do anterior.

Estrutura de pastas sugerida (igual em todos os projetos):

```
projeto-N-nome/  
├─ pom.xml  
├─ docker-compose.yml      (ou usar o compose único do capítulo 8)  
└─ src/  
    └─ main/  
        └─ java/  
            └─ com/estudo/kafka/  
                ├─ Producer.java    (ou nome específico)  
                └─ Consumer.java    (ou nome específico)
```

O domínio usado é o mesmo fio condutor do guia de RabbitMQ — um sistema simples de **e-commerce** (pedidos, pagamentos, estoque, frete) — para que a atenção fique 100% nos conceitos de Kafka, sem gastar energia reinventando o cenário de negócio.

Dependência comum a todos os projetos

Todos usam apenas o cliente oficial `kafka-clients` (sem Spring Kafka), para manter os conceitos explícitos — os mesmos princípios do `amqp-client` puro usado no guia de RabbitMQ.

```
<dependency>  
  <groupId>org.apache.kafka</groupId>  
  <artifactId>kafka-clients</artifactId>  
  <version>3.8.0</version>  
</dependency>
```

Diferença importante em relação ao RabbitMQ: não existe Management UI nativa

O Kafka não vem com uma interface web embutida como o RabbitMQ. O capítulo 8 mostra como adicionar o **Kafka UI** (projeto open source) ao `docker-compose.yml`, para você visualizar tópicos, partições, mensagens e o lag de consumer groups durante os experimentos — o equivalente funcional da Management UI que você já usou.

Sobre o Java 25

O Java 25 é uma LTS muito recente (lançada em setembro de 2025). Se seu ambiente ainda não tiver o JDK 25 disponível, o código do Projeto 5 roda perfeitamente em Java 21 — só perde o uso específico de structured concurrency estabilizada nessa versão.

1. Projeto 1 — Hello World (Java 8)

PROJETO 1 · PRODUTOR/CONSUMIDOR BÁSICO · 1 PARTIÇÃO

pedido-hello-world

Java 8

1.1. Objetivo

Sentir o fluxo mais básico possível: um producer publica um registro em um tópico de partição única, e um consumer o lê. Sem consumer group explícito além do padrão, sem múltiplas partições — só para internalizar `KafkaProducer`, `KafkaConsumer`, `ProducerRecord` e `ConsumerRecords`.

1.2. Estrutura de pastas

```
projeto-1-hello-world/  
├─ pom.xml  
└─ src/main/java/com/estudo/kafka/hello/  
    ├─ PedidoProducer.java  
    └─ PedidoConsumer.java
```

1.3. pom.xml (Java 8)

```
<project xmlns="http://maven.apache.org/POM/4.0.0">  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.estudo</groupId>  
  <artifactId>projeto-1-hello-world</artifactId>  
  <version>1.0.0</version>  
  
  <properties>  
    <maven.compiler.source>8</maven.compiler.source>  
    <maven.compiler.target>8</maven.compiler.target>  
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
  </properties>  
  
  <dependencies>  
    <dependency>  
      <groupId>org.apache.kafka</groupId>  
      <artifactId>kafka-clients</artifactId>  
      <version>3.8.0</version>  
    </dependency>  
  </dependencies>  
</project>
```

1.4. PedidoProducer.java

```
package com.estudo.kafka.hello;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
  
import java.util.Properties;  
  
public class PedidoProducer {  
  
    private static final String TOPIC = "pedidos.simples";  
  
    public static void main(String[] args) throws Exception {  
        Properties props = new Properties();  
        props.put("bootstrap.servers", "localhost:9092");  
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
    }  
}
```

```

KafkaProducer<String, String> producer = new KafkaProducer<String, String>(props);

String mensagem = "Novo pedido: #1001 - Notebook Gamer";

ProducerRecord<String, String> record =
    new ProducerRecord<String, String>(TOPIC, mensagem);

producer.send(record);
producer.flush(); // garante o envio antes de fechar
producer.close();

System.out.println(" [x] Enviado: '" + mensagem + "'");
    }
}

```

1.5. PedidoConsumer.java

```

package com.estudo.kafka.hello;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class PedidoConsumer {

    private static final String TOPIC = "pedidos.simples";

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "grupo-hello-world");
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset", "earliest");

        KafkaConsumer<String, String> consumer = new KafkaConsumer<String, String>(props);
        consumer.subscribe(Collections.singletonList(TOPIC));

        System.out.println(" [*] Aguardando mensagens. CTRL+C para sair.");

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));
            for (ConsumerRecord<String, String> record : records) {
                System.out.println(" [x] Recebido: '" + record.value() + "'"
                    + " (particao=" + record.partition() + ", offset=" + record.offset() + ")");
            }
        }
    }
}

```

1.6. Como rodar

1. Suba o Kafka (veja o capítulo 8 para o `docker-compose.yml` completo).
2. Crie o tópico manualmente (ou deixe o Kafka criar automaticamente na primeira publicação, se `auto.create.topics.enable=true`):

```

kafka-topics.sh --create --topic pedidos.simples \
  --partitions 1 --replication-factor 1 \
  --bootstrap-server localhost:9092

```

3. Rode `PedidoConsumer` primeiro (deixa esperando).
4. Rode `PedidoProducer` em outro terminal.
5. Veja a mensagem chegar no console do consumer, com a partição e o offset atribuídos.

Checklist de aprendizado

- ☐ Entendi que, mesmo sem informar uma chave, o Kafka atribui automaticamente partição e offset ao registro.

- ❑ Rodei `PedidoProducer` três vezes seguidas e vi o offset aumentando (0, 1, 2...) no console do consumer.
- ❑ Parei o consumer, publiquei mais mensagens, e rodei o consumer de novo — confirmei que ele NÃO recebeu as mensagens antigas automaticamente (veja a nota abaixo).

Por que a última mensagem "some" ao reiniciar o consumer

Diferente do RabbitMQ (onde a fila acumula até alguém consumir), aqui o comportamento depende do **commit de offset**. Como não estamos fazendo commit manual neste projeto de aquecimento, o consumer, ao reiniciar com o mesmo `group.id`, retoma de onde parou (se já tiver processado alguma mensagem) ou começa do zero (`earliest`) se nunca processou nada. Offsets e commits são o assunto central do Projeto 2 em diante.

2. Projeto 2 — Partições e Consumer Group (Java 11)

PROJETO 2 · MÚLTIPLAS PARTIÇÕES · PARALELISMO REAL

gerador-relatorios

Java 11

2.1. Objetivo

Ter um tópico com **4 partições** e múltiplas instâncias do mesmo consumer (mesmo `group.id`) competindo pela leitura — o equivalente conceitual ao Work Queue do RabbitMQ, mas com uma diferença fundamental: aqui o paralelismo é limitado ao **número de partições**, não é elástico como no RabbitMQ.

2.2. Estrutura de pastas

```
projeto-2-particoes-consumer-group/  
├─ pom.xml  
└─ src/main/java/com/estudo/kafka/particoes/  
    ├─ RelatorioProducer.java  
    └─ RelatorioWorker.java
```

2.3. pom.xml (Java 11)

```
<properties>  
  <maven.compiler.release>11</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

2.4. Criando o tópico com 4 partições

```
kafka-topics.sh --create --topic fila.relatorios \  
  --partitions 4 --replication-factor 1 \  
  --bootstrap-server localhost:9092
```

2.5. RelatorioProducer.java

```
package com.estudo.kafka.particoes;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
  
import java.util.Properties;  
  
public class RelatorioProducer {  
  
  private static final String TOPIC = "fila.relatorios";  
  
  public static void main(String[] args) {  
    var props = new Properties();  
    props.put("bootstrap.servers", "localhost:9092");  
    props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
    props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
```

```

try (var producer = new KafkaProducer<String, String>(props)) {
    for (int i = 1; i <= 20; i++) {
        String valor = "Relatorio de vendas #" + i;

        // Sem chave (null): o Kafka distribui entre as 4 particoes
        // usando sticky partitioning (por lote, nao round-robin estrito)
        var record = new ProducerRecord<String, String>(TOPIC, valor);

        producer.send(record, (metadata, exception) -> {
            if (exception == null) {
                System.out.println(" [x] Enviado '" + valor + "' -> particao " + metadata.partition());
            }
        });
    }
}
}
}

```

2.6. RelatorioWorker.java

```

package com.estudo.kafka.particoes;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class RelatorioWorker {

    private static final String TOPIC = "fila.relatorios";
    private static final String GROUP_ID = "processamento-relatorios";

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", GROUP_ID);
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("enable.auto.commit", "false");
        props.put("auto.offset.reset", "earliest");

        try (var consumer = new KafkaConsumer<String, String>(props)) {
            consumer.subscribe(Collections.singletonList(TOPIC));

            System.out.println(" [*] Worker pronto. Aguardando particoes serem atribuidas...");

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));

                for (ConsumerRecord<String, String> record : records) {
                    System.out.println(" [x] Particao " + record.partition()
                        + " offset " + record.offset() + " -> " + record.value());

                    // simula trabalho pesado
                    try {
                        Thread.sleep(300 + (long) (Math.random() * 700));
                    } catch (InterruptedException ignored) {}
                }

                if (!records.isEmpty()) {
                    consumer.commitSync();
                }
            }
        }
    }
}

```

2.7. Como rodar

1. Suba o Kafka e crie o tópicos com 4 partições (comando acima).

2. Rode **3 instâncias** de `RelatorioWorker` em terminais separados.
3. Rode `RelatorioProducer` uma vez (publica 20 mensagens).
4. Observe: o Kafka distribui as 4 partições entre os 3 workers — um deles fica com 2 partições, os outros com 1 cada.

Experimento sugerido: o limite do paralelismo

Suba uma **quarta** e uma **quinta** instância de `RelatorioWorker` (5 workers ao todo, 4 partições). Publique mensagens novamente e observe: dois dos cinco workers nunca recebem nada — ficam completamente ociosos, porque não existe partição sobrando para atribuir a eles. Esse é o limite estrutural do paralelismo no Kafka, diferente do RabbitMQ, onde adicionar mais um worker sempre ajuda a dividir a fila.

Checklist de aprendizado

- ☐ Vi, no log do producer, as mensagens sendo distribuídas entre as 4 partições (sem chave, via sticky partitioning).
- ☐ Confirmei que cada partição é lida por exatamente um worker por vez.
- ☐ Subi mais workers do que partições e vi alguns ficarem ociosos.
- ☐ Usei `kafka-consumer-groups.sh --describe --group processamento-relatorios --bootstrap-server localhost:9092` para ver qual worker está lendo qual partição.

3. Projeto 3 — Múltiplos Consumer Groups Independentes (Java 17)

PROJETO 3 · FANOUT NATIVO · CONSUMER GROUPS INDEPENDENTES

pedido-criado-eventos

Java 17

3.1. Objetivo

O equivalente conceitual ao Fanout do RabbitMQ: um evento precisa notificar três serviços independentes (e-mail, estoque, analytics). No Kafka, isso é natural — basta que cada serviço use um `group.id` diferente. Cada grupo lê o tópico inteiro, do seu próprio jeito, com seu próprio offset, sem nenhuma exchange ou binding envolvidos.

3.2. Estrutura de pastas

```
projeto-3-multiplos-consumer-groups/  
├─ pom.xml  
└─ src/main/java/com/estudo/kafka/fanout/  
    ├─ PedidoCriado.java          (record - o evento)  
    ├─ PedidoCriadoProducer.java  
    ├─ EmailService.java  
    ├─ EstoqueService.java  
    └─ AnalyticsService.java
```

3.3. pom.xml (Java 17)

```
<properties>  
  <maven.compiler.release>17</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

3.4. PedidoCriado.java (o record do evento)

```
package com.estudo.kafka.fanout;  
  
import java.math.BigDecimal;  
  
public record PedidoCriado(String pedidoId, String cliente, BigDecimal total) {  
  
  public String toJson() {  
    return ""  
      {"pedidoId":"%s","cliente":"%s","total":%s}""  
      .formatted(  
        pedidoId, cliente, total.toPlainString());  
  }  
}
```

3.5. PedidoCriadoProducer.java

```
package com.estudo.kafka.fanout;
```

```

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;

import java.math.BigDecimal;
import java.util.Properties;

public class PedidoCriadoProducer {

    private static final String TOPIC = "pedidos.criados";

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        try (var producer = new KafkaProducer<String, String>(props)) {
            var evento = new PedidoCriado("PED-2001", "Joana Silva", new BigDecimal("459.90"));

            var record = new ProducerRecord<String, String>(TOPIC, evento.pedidoId(), evento.toJson());

            producer.send(record);
            System.out.println(" [x] Evento publicado: " + evento);
        }
    }
}

```

3.6. Os três consumers (mesmo padrão, group.id diferentes)

```

package com.estudo.kafka.fanout;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class EmailService {

    private static final String TOPIC = "pedidos.criados";
    private static final String GROUP_ID = "email-service"; // <- o pulo do gato

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", GROUP_ID);
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset", "earliest");

        try (var consumer = new KafkaConsumer<String, String>(props)) {
            consumer.subscribe(Collections.singletonList(TOPIC));

            System.out.println(" [*] EmailService aguardando eventos...");

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));
                for (ConsumerRecord<String, String> record : records) {
                    System.out.println(" [email] Enviando confirmacao para: " + record.value());
                }
            }
        }
    }
}

```

EstoqueService e AnalyticsService

São cópias quase idênticas de `EmailService` acima — trocando apenas `GROUP_ID` (ex.: `"estoque-service"`, `"analytics-service"`) e o texto impresso. Cada um desses grupos vai ler o **mesmo tópico inteiro**, de forma totalmente independente — sem precisar declarar filas, exchanges ou bindings, ao contrário do RabbitMQ.

3.7. Como rodar

1. Suba o Kafka e crie o tópico:

```
kafka-topics.sh --create --topic pedidos.criados \  
  --partitions 3 --replication-factor 1 \  
  --bootstrap-server localhost:9092
```

2. Rode os três consumers (`EmailService` , `EstoqueService` , `AnalyticsService`) em terminais separados.
3. Rode `PedidoCriadoProducer` .
4. Veja os **três** serviços reagindo ao mesmo evento, cada um no seu console.

Experimento sugerido: replay do histórico

Pare o `AnalyticsService` . Publique mais 3 eventos com o producer. Agora, em vez de simplesmente religar o `AnalyticsService` , rode-o com `auto.offset.reset=earliest` e um `group.id` **novo** (ex.: `"analytics-service-v2"`) — ele vai ler **todo o histórico do tópico desde o início**, não só os eventos novos. Isso é *replay*, algo que não existe no RabbitMQ: um novo consumer group sempre pode "nascer" lendo tudo que já aconteceu.

3.8. Checklist de aprendizado

- ☐ Vi os três consumer groups recebendo o mesmo evento de forma independente.
- ☐ Testei o replay criando um novo `group.id` e lendo o tópico desde o início.
- ☐ Usei `kafka-consumer-groups.sh --list --bootstrap-server localhost:9092` para ver os três grupos registrados.

4. Projeto 4 — Chaves e Ordenação (Java 21)

PROJETO 4 · PARTICIONAMENTO POR CHAVE · GARANTIA DE ORDEM

pedido-eventos-por-cliente

Java 21

4.1. Objetivo

Provar, na prática, que eventos publicados com a **mesma chave** sempre caem na mesma partição e são lidos **na ordem em que foram publicados** — e que, sem chave, essa garantia desaparece. Usa **virtual threads** (Java 21) para simular múltiplos consumers lendo partições diferentes concorrentemente.

4.2. Estrutura de pastas

```
projeto-4-chaves-ordenacao/  
├── pom.xml  
└── src/main/java/com/estudo/kafka/ordenacao/  
    ├── PedidoEventoProducer.java  
    └── PedidoEventoConsumer.java
```

4.3. pom.xml (Java 21)

```
<properties>  
  <maven.compiler.release>21</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

4.4. PedidoEventoProducer.java

```
package com.estudo.kafka.ordenacao;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
  
import java.util.Properties;  
  
public class PedidoEventoProducer {  
  
  private static final String TOPIC = "pedidos.eventos.por.cliente";  
  
  public static void main(String[] args) throws Exception {  
    var props = new Properties();  
    props.put("bootstrap.servers", "localhost:9092");  
    props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
    props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
  
    try (var producer = new KafkaProducer<String, String>(props)) {  
  
      // Todos os eventos do pedido PED-01 usam a MESMA chave:  
      // garantido cair na mesma particao, na ordem publicada  
      publicar(producer, "PED-01", "CRIADO");  
      publicar(producer, "PED-01", "PAGAMENTO_APROVADO");  
      publicar(producer, "PED-01", "SEPARADO_ESTOQUE");  
      publicar(producer, "PED-01", "ENVIADO");  
    }  
  }  
}
```

```

        // Eventos de um pedido DIFERENTE, chave diferente:
        // pode cair em outra particao, sem relacao de ordem com o PED-01
        publicar(producer, "PED-02", "CRIADO");
        publicar(producer, "PED-02", "PAGAMENTO_RECUSADO");
    }
}

private static void publicar(KafkaProducer<String, String> producer, String pedidoId, String status)
    throws Exception {
    var record = new ProducerRecord<String, String>(
        TOPIC, pedidoId, "{\"status\":\"" + status + "\"}");

    var metadata = producer.send(record).get(); // .get() bloqueia so para fins didaticos
    System.out.println(" [x] " + pedidoId + " -> " + status
        + " (particao=" + metadata.partition() + ", offset=" + metadata.offset() + ")");
    }
}

```

4.5. PedidoEventoConsumer.java (com virtual threads)

```

package com.estudo.kafka.ordenacao;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PedidoEventoConsumer {

    private static final String TOPIC = "pedidos.eventos.por.cliente";

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "auditoria-pedidos");
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset", "earliest");

        // O polling do KafkaConsumer NAO e thread-safe - uma unica thread
        // faz o poll(). Mas o PROCESSAMENTO de cada registro (potencialmente
        // lento: chamadas HTTP, IO) pode ser delegado a virtual threads,
        // sem bloquear o loop principal de poll por muito tempo.
        try (var consumer = new KafkaConsumer<String, String>(props);
            ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {

            consumer.subscribe(Collections.singletonList(TOPIC));
            System.out.println(" [*] Consumer aguardando eventos...");

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));

                for (ConsumerRecord<String, String> record : records) {
                    executor.submit(() -> {
                        System.out.println(" [particao " + record.partition() + "] "
                            + record.key() + " -> " + record.value()
                            + " (offset=" + record.offset() + ")");
                    });
                }
            }
        }
    }
}

```

4.6. Como rodar

1. Suba o Kafka e crie o tópicos com múltiplas partições:

```
kafka-topics.sh --create --topic pedidos.eventos.por.cliente \
  --partitions 3 --replication-factor 1 \
  --bootstrap-server localhost:9092
```

2. Rode `PedidoEventoConsumer`.
3. Rode `PedidoEventoProducer`.
4. Observe no console do consumer: todos os eventos de `PED-01` aparecem na **mesma partição e na ordem publicada** (CRIADO → PAGAMENTO_APROVADO → SEPARADO_ESTOQUE → ENVIADO). Os eventos de `PED-02` podem aparecer em uma partição diferente, intercalados no tempo com os do PED-01.

Experimento sugerido: quebrando a ordem de propósito

Edite o producer para publicar os 4 eventos de `PED-01` usando `ProducerRecord` **sem chave** (passe `null` no lugar de `pedidoId`). Rode novamente e observe no console do consumer: os 4 eventos agora podem cair em partições diferentes e chegar ao consumer **fora de ordem** — a prova definitiva de que a garantia de ordem depende inteiramente da chave.

4.7. Checklist de aprendizado

- ☐ Confirmei que todos os eventos de um mesmo `pedidoId` caem na mesma partição.
- ☐ Vi a ordem de chegada respeitando a ordem de publicação, dentro da mesma chave.
- ☐ Publiquei sem chave (null) e observei a perda da garantia de ordem.

5. Projeto 5 — Falhas, Retry e Reprocessamento (Java 25)

PROJETO 5 · DLQ MANUAL · SEEK · STRUCTURED CONCURRENCY

pagamento-com-retry

Java 25

5.1. Objetivo

Diferente do RabbitMQ, o Kafka **não tem Dead Letter Exchange nativa**. Quando um processamento falha, cabe à sua aplicação decidir o que fazer — geralmente publicando o registro problemático em um **tópico de erro separado** (uma DLQ "manual"). Este projeto implementa esse padrão, junto com uma política de retry com backoff, usando **structured concurrency** (Java 25) para aplicar timeout ao processamento de cada mensagem.

Se não tiver JDK 25 disponível

Toda a lógica de retry/DLQ manual funciona igual em Java 21 LTS — troque `maven.compiler.release` para `21` e substitua o bloco de `StructuredTaskScope` por um `Future.get(timeout)` simples.

5.2. Estrutura de pastas

```
projeto-5-falhas-retry/  
├── pom.xml  
└── src/main/java/com/estudo/kafka/retry/  
    ├── PagamentoProducer.java  
    └── PagamentoConsumer.java
```

5.3. pom.xml (Java 25)

```
<properties>  
  <maven.compiler.release>25</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

5.4. A topologia (tópicos envolvidos)

```
pagamentos.processar (topico principal, o producer publica aqui)  
  |  
  v  
PagamentoConsumer tenta processar  
  |  
  +- sucesso --> commit do offset, segue em frente  
  |  
  +- falha, tentativas < 3 --> republica em pagamentos.processar  
  |                               com header "x-retry-count" incrementado  
  |  
  +- falha, tentativas >= 3 --> publica em pagamentos.dlq  
                               (topico de erro, para investigacao manual)
```

Diferente do RabbitMQ (onde o TTL de uma fila de espera cronometra o atraso automaticamente), aqui **o backoff é responsabilidade do seu código** — não existe um mecanismo nativo de "aguarde X segundos antes de reentregar". A estratégia mais simples (usada abaixo) é um `Thread.sleep` curto antes de republicar; estratégias mais sofisticadas usam

tópicos de espera com nomes como `pagamentos.retry.5s`, `pagamentos.retry.30s`, cada um consumido por um serviço dedicado que aguarda e republica no tópico principal.

5.5. PagamentoProducer.java

```
package com.estudo.kafka.retry;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;

import java.util.Properties;
import java.util.UUID;

public class PagamentoProducer {

    private static final String TOPIC = "pagamentos.processar";

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

        try (var producer = new KafkaProducer<String, String>(props)) {
            for (int i = 1; i <= 5; i++) {
                String pedidoId = "PED-30" + i;
                String payload = "{\"pedidoId\":\"" + pedidoId + "\",\"valor\":199.90}";

                var record = new ProducerRecord<String, String>(TOPIC, pedidoId, payload);
                record.headers().add("x-retry-count", "0".getBytes());
                record.headers().add("x-message-id", UUID.randomUUID().toString().getBytes());

                producer.send(record);
                System.out.println(" [x] Pagamento enviado: " + payload);
            }
        }
    }
}
```

5.6. PagamentoConsumer.java

```
package com.estudo.kafka.retry;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.header.Header;

import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;
import java.util.concurrent.StructuredTaskScope;

public class PagamentoConsumer {

    private static final String TOPIC_PRINCIPAL = "pagamentos.processar";
    private static final String TOPIC_DLQ = "pagamentos.dlq";
    private static final int MAX_TENTATIVAS = 3;

    public static void main(String[] args) {
        var consumerProps = new Properties();
        consumerProps.put("bootstrap.servers", "localhost:9092");
        consumerProps.put("group.id", "processamento-pagamentos");
        consumerProps.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        consumerProps.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        consumerProps.put("enable.auto.commit", "false");
        consumerProps.put("auto.offset.reset", "earliest");

        var producerProps = new Properties();
        producerProps.put("bootstrap.servers", "localhost:9092");
        producerProps.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
    }
}
```

```

producerProps.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

try (var consumer = new KafkaConsumer<String, String>(consumerProps);
    var producer = new KafkaProducer<String, String>(producerProps)) {

    consumer.subscribe(Collections.singletonList(TOPIC_PRINCIPAL));
    System.out.println(" [*] PagamentoConsumer aguardando...");

    while (true) {
        ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));

        for (ConsumerRecord<String, String> record : records) {
            int tentativas = lerTentativas(record) + 1;

            try {
                boolean sucesso = processarComTimeout(record.value());

                if (sucesso) {
                    System.out.println(" [v] Pagamento aprovado: " + record.value());
                } else if (tentativas >= MAX_TENTATIVAS) {
                    enviarParaDlq(producer, record, tentativas);
                } else {
                    reenviarComRetry(producer, record, tentativas);
                }
            } catch (Exception e) {
                enviarParaDlq(producer, record, tentativas);
            }
        }

        if (!records.isEmpty()) {
            consumer.commitSync();
        }
    }
}

// Mesma tecnica de structured concurrency do Projeto 5 de RabbitMQ:
// a subtarefa e o timeout sao tratados como uma unica unidade
private static boolean processarComTimeout(String payload) throws Exception {
    try (var scope = StructuredTaskScope.open(
        StructuredTaskScope.Joiner.<Boolean>anySuccessfulResultOrThrow(),
        cfg -> cfg.withTimeout(java.time.Duration.ofSeconds(2)))) {

        scope.fork(() -> {
            Thread.sleep(300); // simula chamada a um gateway de pagamento
            return Math.random() > 0.4; // ~60% de chance de sucesso
        });

        return scope.join();
    }
}

private static void reenviarComRetry(KafkaProducer<String, String> producer,
                                    ConsumerRecord<String, String> original,
                                    int tentativas) throws InterruptedException {
    // Backoff simples: espera antes de republicar. Em producao,
    // prefira topicos de espera dedicados em vez de bloquear a thread.
    Thread.sleep(2000);

    var record = new ProducerRecord<String, String>(TOPIC_PRINCIPAL, original.key(), original.value());
    record.headers().add("x-retry-count", String.valueOf(tentativas).getBytes());

    producer.send(record);
    System.out.println(" [!] Tentativa " + tentativas + " falhou, reenviado: " + original.value());
}

private static void enviarParaDlq(KafkaProducer<String, String> producer,
                                   ConsumerRecord<String, String> original,
                                   int tentativas) {
    var record = new ProducerRecord<String, String>(TOPIC_DLQ, original.key(), original.value());
    record.headers().add("x-retry-count", String.valueOf(tentativas).getBytes());
    record.headers().add("x-failed-reason", "max-tentativas-excedido".getBytes());

    producer.send(record);
    System.out.println(" [x] Desistindo apos " + tentativas + " tentativas, enviado a DLQ: " + original.value());
}

private static int lerTentativas(ConsumerRecord<String, String> record) {
    Header header = record.headers().lastHeader("x-retry-count");
}

```

```
        if (header == null) return 0;
        return Integer.parseInt(new String(header.value(), StandardCharsets.UTF_8));
    }
}
```

5.7. Como rodar

1. Suba o Kafka e crie os dois tópicos:

```
kafka-topics.sh --create --topic pagamentos.processar --partitions 3 --replication-factor 1 --bootstrap-server localhost:9092
kafka-topics.sh --create --topic pagamentos.dlq --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
```

2. Rode `PagamentoConsumer`.
3. Rode `PagamentoProducer`.
4. Acompanhe no console: mensagens que falham são reenviadas ao mesmo tópico com contador de tentativas incrementado; após 3 tentativas, vão para `pagamentos.dlq`.
5. Inspecione a DLQ manualmente:

```
kafka-console-consumer.sh --topic pagamentos.dlq --from-beginning --bootstrap-server localhost:9092
```

Checklist de aprendizado

- ☐ Vi uma mensagem sendo reenviada ao tópico principal com `x-retry-count` incrementando.
- ☐ Vi uma mensagem chegar até a DLQ depois de esgotar as tentativas.
- ☐ Entendi por que o backoff aqui é responsabilidade da aplicação, diferente do TTL automático do RabbitMQ.
- ☐ Pensei em como implementar tópicos de espera dedicados (`pagamentos.retry.5s`) para não bloquear a thread principal do consumer com `Thread.sleep`.

6. Projeto 6 — Exactly-Once com Transações (Java 21)

PROJETO 6 · READ-PROCESS-WRITE · KAFKA TRANSACTIONS API

pedido-validador-transacional

Java 21

6.1. Objetivo

Implementar o padrão clássico **"ler de um tópico, transformar, escrever em outro"** com garantia *exactly-once*: um serviço lê pedidos "brutos", valida, e escreve no tópico de "validados" — de forma que, mesmo se o processo cair no meio, nenhum pedido seja processado duas vezes nem perdido, e o commit do offset de leitura e a escrita do resultado aconteçam como uma única unidade atômica.

6.2. Estrutura de pastas

```
projeto-6-transacoes/  
├── pom.xml  
└── src/main/java/com/estudo/kafka/transacional/  
    ├── PedidoBrutoProducer.java  
    └── PedidoValidadorTransacional.java
```

6.3. pom.xml (Java 21)

```
<properties>  
  <maven.compiler.release>21</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

6.4. PedidoBrutoProducer.java

```
package com.estudo.kafka.transacional;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
  
import java.util.Properties;  
  
public class PedidoBrutoProducer {  
  
  private static final String TOPIC = "pedidos.brutos";  
  
  public static void main(String[] args) {  
    var props = new Properties();  
    props.put("bootstrap.servers", "localhost:9092");  
    props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
    props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
  
    try (var producer = new KafkaProducer<String, String>(props)) {  
      for (int i = 1; i <= 5; i++) {  
        String pedidoId = "PED-40" + i;  
        String valor = i % 5 == 0 ? "-50.00" : "150.00"; // um valor invalido de proposito  
  
        String payload = "{\"pedidoId\":\"" + pedidoId + "\",\"valor\":\"" + valor + "\"}";  

```

```

        producer.send(new ProducerRecord<String, String>(TOPIC, pedidoId, payload));
        System.out.println(" [x] Pedido bruto enviado: " + payload);
    }
}
}
}

```

6.5. PedidoValidadorTransacional.java

```

package com.estudo.kafka.transacional;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.consumer.OffsetAndMetadata;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.TopicPartition;

import java.time.Duration;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.Properties;

public class PedidoValidadorTransacional {

    private static final String TOPIC_ENTRADA = "pedidos.brutos";
    private static final String TOPIC_SAIDA = "pedidos.validados";

    public static void main(String[] args) {
        var consumerProps = new Properties();
        consumerProps.put("bootstrap.servers", "localhost:9092");
        consumerProps.put("group.id", "validador-pedidos");
        consumerProps.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        consumerProps.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        consumerProps.put("enable.auto.commit", "false");
        consumerProps.put("auto.offset.reset", "earliest");
        // Le somente mensagens de transacoes ja confirmadas (commitadas)
        consumerProps.put("isolation.level", "read_committed");

        var producerProps = new Properties();
        producerProps.put("bootstrap.servers", "localhost:9092");
        producerProps.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        producerProps.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        producerProps.put("transactional.id", "validador-pedidos-tx-1"); // identificador unico e estavel

        try (var consumer = new KafkaConsumer<String, String>(consumerProps);
            var producer = new KafkaProducer<String, String>(producerProps)) {

            producer.initTransactions();
            consumer.subscribe(Collections.singletonList(TOPIC_ENTRADA));

            System.out.println(" [*] Validador transacional aguardando pedidos brutos...");

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));
                if (records.isEmpty()) continue;

                producer.beginTransaction();
                try {
                    Map<TopicPartition, OffsetAndMetadata> offsets = new HashMap<>();

                    for (ConsumerRecord<String, String> record : records) {
                        boolean valido = validar(record.value());
                        String destino = valido ? TOPIC_SAIDA : "pedidos.invalidos";

                        producer.send(new ProducerRecord<String, String>(destino, record.key(), record.value()));
                        System.out.println(" [x] " + record.key() + " -> " + destino);

                        offsets.put(
                            new TopicPartition(record.topic(), record.partition()),
                            new OffsetAndMetadata(record.offset() + 1)
                        );
                    }
                }
            }
        }
    }
}

```

```

        // O commit do offset de LEITURA entra na MESMA transacao
        // da escrita nos topicos de saida - atomico, tudo ou nada
        producer.sendOffsetsToTransaction(offsets, consumer.groupMetadata());
        producer.commitTransaction();

    } catch (Exception e) {
        producer.abortTransaction();
        System.err.println("Transacao abortada: " + e.getMessage());
    }
}

}

private static boolean validar(String payloadJson) {
    // regra fake: valor negativo = invalido
    return !payloadJson.contains("-");
}
}

```

6.6. Como rodar

1. Suba o Kafka e crie os três tópicos:

```

kafka-topics.sh --create --topic pedidos.brutos --partitions 3 --replication-factor 1 --bootstrap-server localhost:9092
kafka-topics.sh --create --topic pedidos.validados --partitions 3 --replication-factor 1 --bootstrap-server
localhost:9092
kafka-topics.sh --create --topic pedidos.invalidos --partitions 1 --replication-factor 1 --bootstrap-server
localhost:9092

```

2. Rode `PedidoValidadorTransacional`.
3. Rode `PedidoBrutoProducer` (publica 5 pedidos, um deles com valor negativo de propósito).
4. Confira os tópicos de saída — 4 pedidos em `pedidos.validados`, 1 em `pedidos.invalidos`.

Checklist de aprendizado

- ☐ Entendi por que `transactional.id` precisa ser único e estável por instância do processo (não gerado aleatoriamente a cada execução).
- ☐ Entendi a diferença entre `isolation.level=read_committed` (só lê o que foi commitado) e `read_uncommitted` (padrão, lê tudo, inclusive de transações abortadas).
- ☐ Testei matar o processo (Ctrl+C) no meio de uma transação e confirmei, ao reiniciar, que nenhum pedido foi processado em duplicidade nem perdido.

7. Projeto 7 — Consumer Produtivo (Java 17)

PROJETO 7 · IDEMPOTÊNCIA · GRACEFUL SHUTDOWN (WAKEUP)

estoque-consumer-robusto

Java 17

7.1. Objetivo

Fechar a progressão com as mesmas duas preocupações de produção do Projeto 7 de RabbitMQ — **idempotência** e **graceful shutdown** — mas usando o mecanismo idiomático do Kafka para desligamento seguro: `consumer.wakeup()`, em vez de um enum de estados controlado manualmente.

7.2. Estrutura de pastas

```
projeto-7-consumer-robusto/  
├── pom.xml  
└── src/main/java/com/estudo/kafka/robusto/  
    ├── RegistroIdempotencia.java  
    └── EstoqueConsumerRobusto.java
```

7.3. pom.xml (Java 17)

```
<properties>  
  <maven.compiler.release>17</maven.compiler.release>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
</properties>  
  
<dependencies>  
  <dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>3.8.0</version>  
  </dependency>  
</dependencies>
```

7.4. RegistroIdempotencia.java

```
package com.estudo.kafka.robusto;  
  
import java.util.Set;  
import java.util.concurrent.ConcurrentHashMap;  
  
public class RegistroIdempotencia {  
  
  // Store em memoria (didatico). Em producao: Redis, banco relacional  
  // com constraint UNIQUE, etc. A chave usada aqui e "particao-offset",  
  // que e naturalmente unica por registro dentro de um topico.  
  private static final Set<String> PROCESSADOS = ConcurrentHashMap.newKeySet();  
  
  public static boolean jaProcessado(int particao, long offset) {  
    return PROCESSADOS.contains(chave(particao, offset));  
  }  
  
  public static void marcarComoProcessado(int particao, long offset) {  
    PROCESSADOS.add(chave(particao, offset));  
  }  
  
  private static String chave(int particao, long offset) {  
    return particao + "-" + offset;  
  }  
}
```

Por que "partição + offset" e não um messageId de negócio

No Kafka, o par (partição, offset) já é um identificador único e natural de cada registro dentro de um tópico — diferente do RabbitMQ, onde não existe um "endereço" tão direto para uma mensagem. Isso simplifica a idempotência quando o objetivo é apenas "nunca reprocessar o mesmo registro físico". Se a idempotência precisar ser por **entidade de negócio** (ex.: nunca aplicar duas baixas de estoque para o mesmo `pedidoId`, mesmo vindas de registros diferentes), aí sim vale usar um identificador de negócio nos headers, como fizemos no `message_id` do RabbitMQ.

7.5. EstoqueConsumerRobusto.java

```
package com.estudo.kafka.robusto;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.errors.WakeupException;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class EstoqueConsumerRobusto {

    private static final String TOPIC = "estoque.baixas";
    private static volatile boolean encerrando = false;

    public static void main(String[] args) {
        var props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "estoque-consumer-robusto");
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("enable.auto.commit", "false");
        props.put("auto.offset.reset", "earliest");

        var consumer = new KafkaConsumer<String, String>(props);

        // Padrao idiomático do Kafka para shutdown gracioso: o shutdown
        // hook chama consumer.wakeup(), que interrompe um poll() em
        // andamento lançando WakeupException - SEM perder o registro
        // que já estava sendo processado, porque o wakeup só afeta
        // a PROXIMA chamada de poll(), não o processamento atual.
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            System.out.println("\n [!] Sinal de shutdown recebido.");
            encerrando = true;
            consumer.wakeup();
        }));

        try {
            consumer.subscribe(Collections.singletonList(TOPIC));
            System.out.println(" [*] EstoqueConsumerRobusto rodando. Ctrl+C para testar o shutdown gracioso.");

            while (!encerrando) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));

                for (ConsumerRecord<String, String> record : records) {
                    if (RegistroIdempotencia.jaProcessado(record.partition(), record.offset())) {
                        System.out.println(" [=] Já processado, ignorando: particao="
                            + record.partition() + " offset=" + record.offset());
                        continue;
                    }

                    System.out.println(" [x] Baixando estoque: " + record.value());
                    simularProcessamento();

                    RegistroIdempotencia.marcarComoProcessado(record.partition(), record.offset());

                    // Commit apos CADA registro processado com sucesso -
                    // garante que, se cair logo em seguida, na pior das
                    // hipoteses reprocessa só este registro (idempotencia
                    // acima cobre esse caso)
                    consumer.commitSync(Collections.singletonMap(
                        new org.apache.kafka.common.TopicPartition(record.topic(), record.partition()),
                        new org.apache.kafka.clients.consumer.OffsetAndMetadata(record.offset() + 1)
                    ));
                }
            }
        } catch (WakeupException e) {
            // Ignorar WakeupException e continuar a consumir
        }
    }
}
```



```

    }
} catch (WakeupException e) {
    // esperado durante o shutdown - nao e um erro real
    System.out.println(" [!] Poll interrompido para shutdown.");
} finally {
    consumer.close(); // libera o consumer do grupo de forma limpa
    System.out.println(" [x] Consumer parado com segurança.");
}
}

private static void simularProcessamento() {
    try {
        Thread.sleep(800);
    } catch (InterruptedException ignored) {
        Thread.currentThread().interrupt();
    }
}
}
}

```

7.6. Como rodar e testar

1. Suba o Kafka e crie o tópico:

```

kafka-topics.sh --create --topic estoque.baixas \
  --partitions 3 --replication-factor 1 \
  --bootstrap-server localhost:9092

```

2. Rode `EstoqueConsumerRobusto`.
3. Publique manualmente pela linha de comando:

```

kafka-console-producer.sh --topic estoque.baixas --bootstrap-server localhost:9092
>{"produtoId":"PROD-42","quantidade":3}

```

7.7. Experimento 1: idempotência automática por natureza

Diferente do RabbitMQ (onde você precisa de um `message_id` explícito controlado pelo producer), aqui a chave de idempotência (`partição + offset`) é atribuída automaticamente pelo próprio Kafka. Para forçar um reprocessamento de teste, use `kafka-consumer-groups.sh` para "voltar" o offset do grupo manualmente:

```

# pausa o consumer antes de rodar isto (Ctrl+C)
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group estoque-consumer-robusto \
  --topic estoque.baixas \
  --reset-offsets --to-earliest --execute

```

Suba o consumer de novo — ele vai reler as mensagens antigas, mas o log vai mostrar `[=] Já processado, ignorando` para cada uma delas, graças ao `RegistroIdempotencia`.

Ressalva importante deste experimento

Como `RegistroIdempotencia` guarda tudo em memória, esse teste só funciona se você **não reiniciar a JVM** entre o processamento original e o reset de offset — reiniciar o processo apaga o registro em memória, e todas as mensagens seriam reprocessadas "de verdade" (o que, inclusive, é um ótimo lembrete de por que produção exige um armazenamento persistente para esse controle).

7.8. Experimento 2: testar o shutdown gracioso

1. Publique uma mensagem nova.
2. Assim que o log `[x] Baixando estoque...` aparecer (durante os 800ms simulados), pressione `Ctrl+C`.
3. Observe: o processamento em andamento **termina normalmente** antes do shutdown hook interromper o próximo `poll()` — a `WakeupException` só afeta a chamada de `poll()` seguinte, nunca o processamento que já estava em curso.

7.9. Checklist de aprendizado

- ☐ Entendi por que `consumer.wakeup()` é o padrão idiomático do Kafka para shutdown, em vez de um enum de estados manual.
- ☐ Testei o reset de offset e confirmei que a idempotência evitou reprocessamento visível no log.
- ☐ Testei o shutdown durante o processamento e confirmei que a mensagem em andamento terminou antes do processo morrer.
- ☐ Pensei em como o par (partição, offset) se compara ao `message_id` manual usado no RabbitMQ.

8. Infraestrutura comum (Docker Compose único)

Em vez de subir um Kafka diferente para cada projeto, use um único broker local para todos os 7 — cada projeto usa tópicos com nomes diferentes, então não há conflito.

8.1. docker-compose.yml (Kafka em modo KRaft + Kafka UI)

```
version: "3.8"
services:
  kafka:
    image: apache/kafka:latest
    container_name: kafka-estudo
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      CLUSTER_ID: Mku3OEVbNTcwNTJENDM2Qk
    volumes:
      - kafka_estudo_data:/var/lib/kafka/data

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    container_name: kafka-ui-estudo
    ports:
      - "8080:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
    depends_on:
      - kafka

volumes:
  kafka_estudo_data:
```

8.2. Comandos básicos

Ação	Comando
Subir o broker	<code>docker compose up -d</code>
Ver logs	<code>docker compose logs -f kafka</code>
Parar	<code>docker compose down</code>
Resetar tudo (apaga tópicos)	<code>docker compose down -v</code>
Kafka UI (equivalente à Management UI do RabbitMQ)	<code>http://localhost:8080</code>

8.3. Executando os scripts de linha de comando via Docker

Se você não tem o Kafka instalado localmente (só via Docker), execute os scripts `kafka-topics.sh` e afins de dentro do container:

```
docker exec -it kafka-estudo kafka-topics.sh --list --bootstrap-server localhost:9092
```

8.4. Organização sugerida do repositório

```
kafka-estudos/
├─ docker-compose.yml
├─ projeto-1-hello-world/
```

```
|— projeto-2-particoes-consumer-group/  
|— projeto-3-multiplos-consumer-groups/  
|— projeto-4-chaves-ordenacao/  
|— projeto-5-falhas-retry/  
|— projeto-6-transacoes/  
|— projeto-7-consumer-robusto/
```

8.5. Ordem de execução recomendada

#	Projeto	Java	Conceito novo
1	Hello World	8	Producer/consumer básico, partição única
2	Partições e Consumer Group	11	Paralelismo real, limite = número de partições
3	Múltiplos Consumer Groups	17	Fanout nativo, replay de histórico
4	Chaves e Ordenação	21	Particionamento por chave, garantia de ordem
5	Falhas, Retry e Reprocessamento	25	DLQ manual, backoff sem mecanismo nativo
6	Exactly-Once com Transações	21	Read-process-write atômico
7	Consumer Produtivo	17	Idempotência + graceful shutdown (wakeup)

8.6. Comparando com a jornada de RabbitMQ

RabbitMQ	Kafka (equivalente conceitual)
Projeto 1 — Hello World	Projeto 1 — Hello World
Projeto 2 — Work Queue (workers competindo)	Projeto 2 — Partições + Consumer Group
Projeto 3 — Fanout (múltiplas filas)	Projeto 3 — Múltiplos Consumer Groups
Projeto 4 — Topic Exchange (roteamento)	Projeto 4 — Chaves e Ordenação (mecanismo diferente, mesma preocupação com "quem recebe o quê")
Projeto 5 — DLX + Retry (nativo)	Projeto 5 — DLQ manual + Retry (você implementa)
Projeto 6 — RPC síncrono	Projeto 6 — Transações exactly-once (preocupação diferente: não existe RPC nativo no Kafka)
Projeto 7 — Consumer Robusto	Projeto 7 — Consumer Robusto

Nota sobre o Projeto 6

Vale destacar que o Projeto 6 é o ponto onde a analogia com RabbitMQ mais se quebra: o Kafka não tem um padrão nativo de RPC síncrono (não faria muito sentido em um sistema pensado para logs de eventos de alto throughput). Em vez disso, usamos esse projeto para cobrir um conceito que é exclusivamente relevante no mundo Kafka: **transações** — um mecanismo sem equivalente direto no guia de RabbitMQ.

Próximo passo

Depois de rodar os 7 projetos, você terá praticado os principais conceitos do guia teórico de Kafka. Se quiser aprofundar ainda mais, os próximos passos naturais seriam explorar o **Kafka Streams** (processamento contínuo sobre tópicos) ou montar um guia visual com diagramas de arquitetura e fluxogramas para cada um destes 7 projetos — no mesmo formato do guia visual que você já tem de RabbitMQ.